

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт

з лабораторної роботи № 1 з дисципліни
«Проектування алгоритмів»

„Проектування і аналіз алгоритмів зовнішнього сортування”

Виконав

ІП-341 Черепов Олександр Павлович
(шифр, прізвище, ім'я, по батькові)

Перевірив

Головченко М.М. _____
(прізвище, ім'я, по батькові)

Київ 2025

ЗМІСТ

1	МЕТА ЛАБОРАТОРНОЇ РОБОТИ	3
2	ЗАВДАННЯ	4
3	ВИКОНАННЯ.....	5
3.1	ПСЕВДОКОД АЛГОРИТМУ	5
3.2	ПРОГРАМНА РЕАЛІЗАЦІЯ АЛГОРИТМУ	12
3.2.1	<i>Вихідний код.....</i>	<i>12</i>
3.2.2	<i>Запити до чат-боту.....</i>	<i>18</i>
	ВИСНОВОК	25
	КРИТЕРІЇ ОЦІНЮВАННЯ	25

1 МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Мета роботи – вивчити основні алгоритми зовнішнього сортування та способи їх модифікації, оцінити поріг їх ефективності.

2 ЗАВДАННЯ

Згідно варіанту (таблиця 2.1), розробити та записати алгоритм зовнішнього сортування за допомогою псевдокоду (чи іншого способу за вибором).

Виконати програмну реалізацію алгоритму на будь-якій мові програмування та відсортувати випадковим чином згенерований файл за ключем (розмір файлу має бути не менше 10 Мб, можна значно більше).

Структура файлу наступна:

- **задача А** число (ключ) - рядок довжиною до 20 символів – дата;
- **задача В** латинська літера (ключ) - рядок довжиною до 45 символів - номер телефону.

Здійснити модифікацію програми і відсортувати файл за ключем. Досягти швидкості сортування з розрахунку 1Гб на 3-5хв. або менше. Необхідно штучно обмежити доступну ОП у відповідності із варіантом (наприклад використовуючи віртуальну машину, налаштування IDE тощо).

Рекомендується попередньо впорядкувати серії елементів довжиною, що займає не менше 100Мб або використати інші підходи для пришвидшення процесу сортування.

За допомогою чат-боту (на власний вибір) створити модифіковану версію програми сортування файлу дослідити її, і вказати переваги та недоліки. Алгоритм сортування має відрізнятись від лекційного але обмеження взяти відповідно до варіанту.

Зробити узагальнений висновок з лабораторної роботи, у якому порівняти базову та модифіковані програми. У висновку деталізувати, які саме модифікації було виконано і який ефект вони дали. Описати переваги і недоліки кожної версії.

Номер та завдання варіанту:

22	Природне (адаптивне) злиття	А	зростання	300МБ
----	--------------------------------	---	-----------	-------

3 ВИКОНАННЯ

3.1 Псевдокод алгоритму

Початкова версія алгоритму:

Загальний алгоритм:

```
procedure SortAdaptiveAsc(mainFile, tempFile0, tempFile1)
    while IsSorted(mainFile) is false do
        DistributeData(mainFile, tempFile0, tempFile1)
        MergeFiles(mainFile, tempFile0, tempFile1)
    end while

    delete file tempFile0
    delete file tempFile1
end procedure
```

Розподіл серій:

```
procedure DistributeData(mainFile, tempFile0, tempFile1)
    previousKey = -infinity
    fileIndex = 0

    main_f = open mainFile for reading
    f0 = open tempFile0 for writing
    f1 = open tempFile1 for writing
    tempFiles = [f0, f1]

    for each line in main_f do
        currentKey = extractKey(line)
        if previousKey > currentKey then
            fileIndex = fileIndex + 1
        end if
        writeToFile(tempFiles[fileIndex mod 2], line)
        previousKey = currentKey
    end for
```

```
        close all files
    end procedure
```

Злиття файлів:

```
procedure MergeFiles(mainFile, tempFile0, tempFile1)

    f0 = open tempFile0 for reading
    f1 = open tempFile1 for reading
    main_f = open mainFile for writing

    line0 = readLine(f0)
    line1 = readLine(f1)

    while line0 is not empty AND line1 is not empty do
        end0 = False
        end1 = False

        while end0 is False AND end1 is False do
            key0 = extractKey(line0)
            key1 = extractKey(line1)

            if key0 <= key1 then
                writeToFile(main_f, line0)
                prev = key0
                line0 = readLine(f0)

                if line0 is empty OR extractKey(line0) < prev then
                    end0 = True
                end if
            else
                writeToFile(main_f, line1)
                prev = key1
            end if
        end while
    end while
end procedure
```

```

        line1 = readLine(f1)

        if line1 is empty OR extractKey(line1) < prev then
            end1 = True
        end if
    end if
end while

while end0 is False do
    writeToFile(main_f, line0)
    prev = extractKey(line0)
    line0 = readLine(f0)

    if line0 is empty OR extractKey(line0) < prev then
        end0 = True
    end if
end while

while end1 is False do
    writeToFile(main_f, line1)
    prev = extractKey(line1)
    line1 = readLine(f1)

    if line1 is empty OR extractKey(line1) < prev then
        end1 = True
    end if
end while
end while

while line0 is not empty do
    writeToFile(main_f, line0)
    line0 = readLine(f0)

```

```

end while

while line1 is not empty do
    writeToFile(main_f, line1)
    line1 = readLine(f1)
end while

close all files
end procedure

```

Модифікована версія алгоритму:

Загальний алгоритм:

```

procedure SortAdaptiveAsc(mainFile)
    chunkFiles = SplitIntoSortedChunks(mainFile)

    MergeChunks(chunkFiles, mainFile)

    for each file in chunkFiles do
        delete file
    end for
end procedure

```

Розбиття на чанки:

```

procedure SplitIntoSortedChunks(inputFile)
    chunkFiles = empty list
    chunkData = empty list
    currentSize = 0
    chunkIndex = 0

    input_f = open inputFile for reading

    while input_f has lines do

```



```

    line = readLine(input_f)
    append line to chunkData
    currentSize = currentSize + length(line)

    if currentSize >= RAM_LIMIT then
        Sort(chunkData) by key ascending

        tempFileName = "chunk_" + chunkIndex
        tempFile = open tempFileName for writing
        writeToFile(tempFile, chunkData)
        close tempFile

        append tempFileName to chunkFiles

        clear chunkData
        currentSize = 0
        chunkIndex = chunkIndex + 1
    end if
end while

if chunkData is not empty then
    Sort(chunkData) by key ascending
    tempFileName = "chunk_" + chunkIndex
    tempFile = open tempFileName for writing
    writeToFile(tempFile, chunkData)
    close tempFile

    append tempFileName to chunkFiles
end if
close input_f
return chunkFiles
end procedure

```

Злиття файлів:

```
procedure MergeChunks(chunkFiles, outputFile)
    openFiles = empty list
    currentLines = empty list

    for each filename in chunkFiles do
        f = open filename for reading
        line = readLine(f)
        if line is not empty then
            append f to openFiles
            key = extractKey(line)
            append (key, line) to currentLines
        else
            close f
        end if
    end for

    out_f = open outputFile for writing

    while currentLines is not empty do
        minIndex = 0
        minValue = currentLines[0].key

        for i from 1 to length(currentLines) - 1 do
            if currentLines[i].key < minValue then
                minValue = currentLines[i].key
                minIndex = i
            end if
        end for

        writeToFile(out_f, currentLines[minIndex].line)
```

```

    originFile = openFiles[minIndex]
    nextLine = readLine(originFile)

    if nextLine is not empty then
        nextKey = extractKey(nextLine)
        currentLines[minIndex] = (nextKey, nextLine)
    else
        close originFile
        remove index minIndex from openFiles
        remove index minIndex from currentLines
    end if
end while

close out_f
end procedure

```

3.2 Програмна реалізація алгоритму

3.2.1 Вихідний код

main.py:

```
from generate import generate_data
import time
import math
import os

FILES_BASE_PATH = "./files"

def extract_key_from_line(line):
    return int(line.partition(' ')[0])

def format_time(seconds):
    time_output = ""
    hours = seconds // 3600
    minutes = (seconds % 3600) // 60
    seconds_left = seconds % 60

    if hours >= 1:
        time_output += f"{hours} hours, "

    if minutes >= 1:
        time_output += f"{minutes} minutes, "

    time_output += f"{round(seconds_left, 2)} seconds"

    return time_output

def merge_files(main_file, file0="0.txt", file1="1.txt"):
    with open(f"{FILES_BASE_PATH}/{file0}", 'r') as f0, \
        open(f"{FILES_BASE_PATH}/{file1}", 'r') as f1, \
        open(f"{FILES_BASE_PATH}/{main_file}", 'w') as main_f:

        line0 = f0.readline()
        line1 = f1.readline()

        while line0 and line1:
            end0 = False
            end1 = False

            while not end0 and not end1:
                key0 = extract_key_from_line(line0)
                key1 = extract_key_from_line(line1)
```

```

        if key0 <= key1:
            main_f.write(line0)
            line0 = f0.readline()

            if not line0 or extract_key_from_line(line0) < key0:
                end0 = True
        else:
            main_f.write(line1)
            line1 = f1.readline()

            if not line1 or extract_key_from_line(line1) < key1:
                end1 = True

    while not end0:
        main_f.write(line0)
        prev = extract_key_from_line(line0)
        line0 = f0.readline()

        if not line0 or extract_key_from_line(line0) < prev:
            end0 = True

    while not end1:
        main_f.write(line1)
        prev = extract_key_from_line(line1)
        line1 = f1.readline()

        if not line1 or extract_key_from_line(line1) < prev:
            end1 = True

    while line0:
        main_f.write(line0)
        line0 = f0.readline()

    while line1:
        main_f.write(line1)
        line1 = f1.readline()

def distribute_data(filename, file0="0.txt", file1="1.txt"):
    prev = - (math.inf)
    file_flag = 0
    chunks = 0

    with open(f"{FILES_BASE_PATH}/{filename}", "r") as main_f, \
        open(f"{FILES_BASE_PATH}/{file0}", "w") as f0, \
        open(f"{FILES_BASE_PATH}/{file1}", "w") as f1:

        temp_files = [f0, f1]

        for line in main_f:
            cur = extract_key_from_line(line)

```

```

        if prev > cur:
            file_flag += 1
            chunks += 1

        temp_files[file_flag % 2].write(line)

        prev = cur

    return chunks

def sort_adaptive_asc(filename, file0="0.txt", file1="1.txt",
file_size_mb=10, max_string_len=20):
    generate_data(f"{FILES_BASE_PATH}/{filename}", file_size_mb,
max_string_len)

    start_time = time.time()
    print(f"STARTING SORTING PROCESS... [{start_time}]")

    while True:
        chunks = distribute_data(filename)

        if chunks > 0: merge_files(filename)
        else: break

    os.remove(f"{FILES_BASE_PATH}/{file0}")
    os.remove(f"{FILES_BASE_PATH}/{file1}")

    end_time = time.time()

    print(f"SORTING EXECUTED SUCCESSFULLY [{end_time}]")
    print("PROCESSING TIME:", format_time(end_time - start_time),
f"[{end_time - start_time} seconds]")

```

main_enhanced.py:

```

from generate import generate_data
import time
import os

FILES_BASE_PATH = "./files"
CHUNK_SIZE_BYTES = 25 * 1024 * 1024

def extract_key_from_line(line):
    return int(line.partition(' ')[0])

def format_time(seconds):

```

```

time_output = ""
hours = seconds // 3600
minutes = (seconds % 3600) // 60
seconds_left = seconds % 60

if hours >= 1:
    time_output += f"{int(hours)} hours, "

if minutes >= 1:
    time_output += f"{int(minutes)} minutes, "

time_output += f"{round(seconds_left, 2)} seconds"

return time_output

def split_into_sorted_chunks(input_file):
    chunk_files = []

    chunk_data = []
    curr_size = 0
    chunk_index = 0

    with open(f"{FILES_BASE_PATH}/{input_file}", "r") as file:
        for line in file:
            chunk_data.append(line)
            curr_size += len(line)

            if curr_size >= CHUNK_SIZE_BYTES:
                chunk_data.sort(key=extract_key_from_line)

                chunk_filename = f"{FILES_BASE_PATH}/chunk{chunk_index}.txt"

                with open(chunk_filename, "w") as chunk_file:
                    chunk_file.writelines(chunk_data)

                chunk_files.append(chunk_filename)

                chunk_data = []
                curr_size = 0
                chunk_index += 1

    if chunk_data:
        chunk_data.sort(key=extract_key_from_line)
        chunk_filename = f"{FILES_BASE_PATH}/chunk{chunk_index}.txt"

        with open(chunk_filename, 'w') as chunk_f:
            chunk_f.writelines(chunk_data)

        chunk_files.append(chunk_filename)

```

```

    return chunk_files

def merge_chunks(chunk_files, output_file):
    files = []
    current_lines = []

    try:
        for filename in chunk_files:
            f = open(filename, "r")
            line = f.readline()
            if line:
                files.append(f)
                current_lines.append((extract_key_from_line(line), line))
            else:
                f.close()

        with open(f"{FILES_BASE_PATH}/{output_file}", "w") as out_file:
            while current_lines:
                min_index = 0
                min_val = current_lines[0][0]

                for i in range(1, len(current_lines)):
                    if current_lines[i][0] < min_val:
                        min_val = current_lines[i][0]
                        min_index = i

                _, line_to_write = current_lines[min_index]
                out_file.write(line_to_write)

                next_line = files[min_index].readline()

                if next_line:
                    current_lines[min_index] =
(extract_key_from_line(next_line), next_line)
                else:
                    files[min_index].close()
                    files.pop(min_index)
                    current_lines.pop(min_index)

            finally:
                for f in files:
                    f.close()

def sort_adaptive_asc(filename, file_size_mb=1000, max_string_len=20):
    generate_data(f"{FILES_BASE_PATH}/{filename}", file_size_mb,
max_string_len)

    start_time = time.time()
    print(f"STARTING SORTING PROCESS... [{start_time}]")

```



```

chunk_files = split_into_sorted_chunks(filename)
merge_chunks(chunk_files, filename)

for f in chunk_files:
    if os.path.exists(f):
        os.remove(f)

end_time = time.time()

print(f"SORTING EXECUTED SUCCESSFULLY [{end_time}]")
print("PROCESSING TIME:", format_time(end_time - start_time))

```

generate.py:

```

import random
import string
import calendar
from datetime import date

START_YEAR = 1
END_YEAR = 2077

def generate_random_date():
    year = random.randint(START_YEAR, END_YEAR)
    month = random.randint(1, 12)

    if month == 2:
        if calendar.isleap(year):
            day = random.randint(1, 29)
        else:
            day = random.randint(1, 28)
    elif month in [4, 6, 9, 11]:
        day = random.randint(1, 30)
    else:
        day = random.randint(1, 31)

    return date(year, month, day)

def generate_data(file, file_size_mb, max_string_len):
    print(f"GENERATING FRESH DATA [Size: {file_size_mb} MB, Max string length: {max_string_len}]")

    with open(file, "w") as f:
        size = 0
        lines = 0
        target = file_size_mb * 1024 * 1024

        while size < target:

```

```

        rand_num = str(random.randint(0, 99999))
        rand_string = ''.join(random.choices(string.ascii_letters +
string.digits, k=random.randint(1, max_string_len)))
        rand_date = generate_random_date()

        line = f"{rand_num} - {rand_string} -
{rand_date.strftime('%d.%m.%Y')}\n"

        f.write(line)
        size += len(line)
        lines += 1

    print(f"FILE DATA GENERATED SUCCESSFULLY [{lines} lines]")

```

3.2.2 Запити до чат-боту (Модель AI: Claude Sonnet 4.5; Free plan)

Занум:

i need you to develop an adaptive external sorting algorithm that minimizes RAM usage and keeps it in 300 MB limit, uses only 2 temporary files for sorting data and has capacity of sorting 1 GB of data in about 3-5 minutes. incoming data example: large .txt file that contains several automatically generated lines following this schema:

random number (key) - random string (20 chars max) - random date

the data should be sorted by the key (number) in ascending order and written back to the original file.

use Python for completing this task.

Відповідь:

```

import os
import tempfile
import heapq
from typing import List, Tuple, TextIO
import math

class AdaptiveExternalSort:
    def __init__(self, max_memory_mb: int = 250):
        """

```

```

Initialize the external sorter.

Args:
    max_memory_mb: Maximum memory to use in MB (default 250MB,
leaving buffer)
    """
    self.max_memory_bytes = max_memory_mb * 1024 * 1024
    self.temp_file1 = None
    self.temp_file2 = None

def parse_line(self, line: str) -> Tuple[int, str]:
    """Parse a line and return (key, original_line) tuple."""
    key = int(line.split(' - ')[0])
    return (key, line.rstrip('\n'))

def estimate_lines_per_chunk(self, filepath: str) -> int:
    """
    Estimate how many lines can fit in memory based on file size.
    """
    # Sample first 1000 lines to estimate average line size
    sample_size = 0
    line_count = 0

    with open(filepath, 'r', encoding='utf-8') as f:
        for i, line in enumerate(f):
            if i >= 1000:
                break
            sample_size += len(line.encode('utf-8'))
            line_count += 1

    if line_count == 0:
        return 1000

    avg_line_size = sample_size / line_count
    # Account for tuple overhead and parsed data structure
    # Each line becomes (int, str) tuple with additional overhead
    memory_per_line = avg_line_size + 100 # Add overhead for structures

    lines_per_chunk = int(self.max_memory_bytes / memory_per_line)
    return max(1000, lines_per_chunk) # Minimum 1000 lines per chunk

def create_sorted_chunks(self, input_file: str) -> Tuple[int, int]:
    """
    Phase 1: Read input file in chunks, sort each chunk, and write to
temp files.
    Returns (number_of_chunks, lines_per_chunk).
    """
    lines_per_chunk = self.estimate_lines_per_chunk(input_file)
    chunk_count = 0

```

```

        # Create temporary files
        temp_dir = tempfile.gettempdir()
        self.temp_file1 = os.path.join(temp_dir,
f'sort_temp1_{os.getpid()}.txt')
        self.temp_file2 = os.path.join(temp_dir,
f'sort_temp2_{os.getpid()}.txt')

        with open(input_file, 'r', encoding='utf-8') as infile, \
            open(self.temp_file1, 'w', encoding='utf-8') as outfile:

            while True:
                # Read chunk into memory
                chunk = []
                for _ in range(lines_per_chunk):
                    line = infile.readline()
                    if not line:
                        break
                    chunk.append(self.parse_line(line))

                if not chunk:
                    break

                # Sort chunk in memory
                chunk.sort(key=lambda x: x[0])

                # Write sorted chunk with chunk marker
                outfile.write(f"#CHUNK#{chunk_count}\n")
                for _, original_line in chunk:
                    outfile.write(original_line + '\n')

                chunk_count += 1
                chunk.clear() # Free memory

            return chunk_count, lines_per_chunk

def merge_chunks(self, num_chunks: int, output_file: str):
    """
    Phase 2: Merge sorted chunks using two-way merge with heap.
    Uses alternating temp files to minimize disk usage.
    """
    if num_chunks == 0:
        return

    if num_chunks == 1:
        # Only one chunk, just copy it back
        self._copy_chunk_to_output(self.temp_file1, 0, output_file)
        return

    current_input = self.temp_file1
    current_output = self.temp_file2

```

```

chunks_remaining = num_chunks
merge_level = 0

while chunks_remaining > 1:
    merged_chunks = 0

    with open(current_output, 'w', encoding='utf-8') as outfile:
        chunk_idx = 0

        while chunk_idx < chunks_remaining:
            # Merge two consecutive chunks at a time
            chunk1_data = list(self._read_chunk(current_input,
chunk_idx))

            chunk2_data = []

            if chunk_idx + 1 < chunks_remaining:
                chunk2_data = list(self._read_chunk(current_input,
chunk_idx + 1))

            # Merge using heap
            outfile.write(f"#CHUNK#{merged_chunks}\n")
            self._merge_two_chunks(chunk1_data, chunk2_data, outfile)

            chunk_idx += 2
            merged_chunks += 1

        # Swap input and output files
        current_input, current_output = current_output, current_input
        chunks_remaining = merged_chunks
        merge_level += 1

    # Final chunk is in current_input, copy to output
    self._copy_chunk_to_output(current_input, 0, output_file)

def _read_chunk(self, filepath: str, chunk_index: int) -> List[Tuple[int,
str]]:
    """Read a specific chunk from a file."""
    with open(filepath, 'r', encoding='utf-8') as f:
        current_chunk = -1
        for line in f:
            if line.startswith('#CHUNK#'):
                current_chunk = int(line.strip().split('#')[2])
                if current_chunk > chunk_index:
                    break
                continue

            if current_chunk == chunk_index:
                yield self.parse_line(line)

```

```

def _merge_two_chunks(self, chunk1: List[Tuple[int, str]],
                      chunk2: List[Tuple[int, str]], outfile: TextIO):
    """Merge two sorted chunks using heap."""
    i, j = 0, 0
    len1, len2 = len(chunk1), len(chunk2)

    while i < len1 and j < len2:
        if chunk1[i][0] <= chunk2[j][0]:
            outfile.write(chunk1[i][1] + '\n')
            i += 1
        else:
            outfile.write(chunk2[j][1] + '\n')
            j += 1

    # Write remaining elements
    while i < len1:
        outfile.write(chunk1[i][1] + '\n')
        i += 1

    while j < len2:
        outfile.write(chunk2[j][1] + '\n')
        j += 1

def _copy_chunk_to_output(self, temp_file: str, chunk_index: int,
                          output_file: str):
    """Copy a specific chunk from temp file to output."""
    with open(temp_file, 'r', encoding='utf-8') as infile, \
        open(output_file, 'w', encoding='utf-8') as outfile:

        current_chunk = -1
        for line in infile:
            if line.startswith('#CHUNK#'):
                current_chunk = int(line.strip().split('#')[2])
                continue

            if current_chunk == chunk_index:
                outfile.write(line)

def sort_file(self, input_file: str):
    """
    Main sorting function. Sorts the file in-place.

    Args:
        input_file: Path to the input file to sort
    """
    try:
        print(f"Starting external sort of {input_file}")
        print(f"Memory limit: {self.max_memory_bytes / (1024*1024):.0f}
MB")

```

```

        # Phase 1: Create sorted chunks
        print("Phase 1: Creating sorted chunks...")
        num_chunks, lines_per_chunk =
self.create_sorted_chunks(input_file)
        print(f"Created {num_chunks} chunks ({lines_per_chunk} lines per
chunk)")

        # Phase 2: Merge chunks
        print("Phase 2: Merging chunks...")
        self.merge_chunks(num_chunks, input_file)
        print("Sort complete!")

    finally:
        # Cleanup temporary files
        self._cleanup()

def _cleanup(self):
    """Remove temporary files."""
    for temp_file in [self.temp_file1, self.temp_file2]:
        if temp_file and os.path.exists(temp_file):
            try:
                os.remove(temp_file)
            except:
                pass

# Example usage
if __name__ == "__main__":
    import time

    # Create a test file (optional - for testing)
    def create_test_file(filename: str, num_lines: int = 100000):
        """Create a test file with random data."""
        import random
        import string
        from datetime import datetime, timedelta

        print(f"Creating test file with {num_lines} lines...")
        with open(filename, 'w') as f:
            for _ in range(num_lines):
                key = random.randint(1, 1000000)
                text = ''.join(random.choices(string.ascii_letters,
k=random.randint(5, 20)))
                days_ago = random.randint(0, 365)
                date = (datetime.now() -
timedelta(days=days_ago)).strftime('%Y-%m-%d')
                f.write(f"{key} - {text} - {date}\n")
        print(f"Test file created: {filename}")

    # Example: Sort a file
    test_file = "data_to_sort.txt"

```

```
# Uncomment to create a test file:
# create_test_file(test_file, 1000000) # 1 million lines (~70MB)

# Sort the file
sorter = AdaptiveExternalSort(max_memory_mb=250)

start_time = time.time()
sorter.sort_file(test_file)
end_time = time.time()

print(f"\nSorting completed in {end_time - start_time:.2f} seconds")
```


ВИСНОВОК

У ході лабораторної роботи було досліджено проблему зовнішнього сортування великих масивів даних та реалізовано різні підходи до її вирішення. Спочатку було розглянуто стандартний алгоритм природного злиття. Тестування показало, що хоча він є ефективним для частково впорядкованих послідовностей, на повністю випадкових наборах даних він демонструє низьку продуктивність через надмірну кількість операцій читання та запису на диск, що не дозволило виконати часові нормативи.

Для досягнення необхідної швидкості та дотримання обмежень пам'яті було розроблено модифіковану версію програми. Алгоритм змінив стратегію на розбиття файлу на чанки розміром 25 МБ, які сортуються безпосередньо в оперативній пам'яті перед записом у тимчасові файли. Це дозволило мінімізувати кількість дискових операцій, вкластися в ліміт 300 МБ RAM та успішно відсортувати 1 ГБ даних за приблизно 5 хвилин.

Окремо було проаналізовано версію алгоритму, запропоновану штучним інтелектом. AI-рішення реалізувало аналогічний підхід із фіксованими блоками, але з використанням об'єктно-орієнтованого стилю та модуля heapq для оптимізації злиття. Порівняння показало, що хоча логіка роботи модифікованої та AI-версій схожа, ШІ запропонував більш структуровану архітектуру.

КРИТЕРІЇ ОЦІНЮВАННЯ

Максимальний бал за виконання лабораторної роботи дорівнює – 5.

Критерії оцінювання:

- проектування алгоритму – 1;
- програмна реалізація алгоритму – 1;
- захист – 1;
- дослідження алгоритму – 1;
- звіт – 1.